GitHub Actions YAML Cheat Sheet

Practical syntax + examples for CI/CD, Docker, AWS, Terraform and DevSecOps

> **Use this PDF when writing or debugging** `.github/workflows/*.yml`**.**

Focus: the YAML structures and GitHub Actions patterns that appear most often in cloud engineering and cloud security projects.

Section	What you get
1. YAML basics	Indentation, lists, maps, strings, booleans, multiline blocks
2. Workflow anatomy	name, on, permissions, env, concurrency, jobs, steps
3. Expressions	${{ ... }}, contexts, conditions, functions
4. Data passing	env, vars, secrets, outputs, artifacts
5. CI patterns	Python, Docker, Trivy, Terraform, Checkov
6. AWS patterns	OIDC, ECR, Terraform plan/apply, ECS verification
7. Advanced patterns	matrix, reusable workflows, environments, caching
8. Debugging	Common YAML and Actions mistakes

1. YAML Basics You Need for GitHub Actions

Indentation

YAML uses indentation to show structure. Use spaces, not tabs. Two spaces is the most common style.

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4
```

> **Rule: items at the same logical level must have the same indentation.**

Maps (key: value)

```
name: CI Pipeline
runs-on: ubuntu-latest
timeout-minutes: 10
```

Lists

```
branches:
  - main
  - develop

needs:
  - test
  - security
```

List of objects

```
steps:
  - name: Checkout Repository
    uses: actions/checkout@v4

  - name: Run Tests
    run: pytest
```

Strings

Syntax	Example	Use
Plain	name: Python Tests	Normal text
Single quotes	if: 'success()'	Protect special characters
Double quotes	python-version: "3.12"	Explicit string
Block |	run: |	Multiline shell commands
Folded >	description: >	Multiline text folded into one line

Multiline shell commands

```
- name: Terraform Init
  run: |
    terraform init       -reconfigure       -input=false
```

Comments

```
# This job runs unit tests.
```

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
```

2. Workflow Anatomy

```
name: CI/CD Pipeline

on:
  push:
    branches:
      - main

permissions:
  contents: read
  id-token: write

env:
  PYTHON_VERSION: "3.12"

concurrency:
  group: deploy-${{ github.ref }}
  cancel-in-progress: false

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Run Test
        run: echo "hello"
```

Key	Purpose
name	Workflow name shown in GitHub Actions
on	Events that start the workflow
permissions	GitHub token/OIDC permissions
env	Workflow-level environment variables
concurrency	Controls overlapping workflow runs
jobs	Independent units of work
runs-on	Runner operating system
steps	Commands/actions inside a job
uses	Run a reusable GitHub Action
run	Execute shell commands

Common triggers

```
on:
  push:
    branches:
      - main

  pull_request:
    branches:
      - main

  workflow_dispatch:
```

Trigger only for certain paths

```
on:
  push:
    branches: [main]
    paths:
      - "app/**"
      - "docker/**"
      - ".github/workflows/**"
```

Scheduled workflow

```
on:
  schedule:
    - cron: "0 8 * * 1"
```

GitHub cron uses UTC. The example runs every Monday at 08:00 UTC.

3. Jobs, Steps, needs and if

Basic job

```
jobs:
  test:
    name: Python Tests
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Run Tests
        run: pytest
```

Job dependency with needs

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - run: pytest

  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying"
```

By default, `deploy` runs only if `test` succeeds.

Depend on several jobs

```
needs:
  - test
  - repository-security
  - terraform
```

Job condition

```
if: github.ref == 'refs/heads/main' && github.event_name == 'push'
```

Step conditions

```
- name: Upload Logs
  if: failure()
  run: ./collect-logs.sh

- name: Cleanup
  if: always()
  run: docker rm -f app || true

- name: Success Message
  if: success()
  run: echo "Passed"
```

Function	Meaning
success()	Previous required work succeeded
failure()	A previous step/job failed
always()	Run even after success/failure/cancel

Function	Meaning
cancelled()	Run if workflow was cancelled

4. Expressions and Contexts

GitHub Actions expressions are written inside `${{ ... }}`.

```
${{ github.sha }}
${{ github.ref }}
${{ vars.AWS_REGION }}
${{ secrets.MY_SECRET }}
${{ steps.build.outputs.image }}
${{ needs.push-ecr.outputs.image }}
```

Most useful contexts

Context	Example	Meaning
github	${{ github.sha }}	Commit, ref, event, actor, repository
env	${{ env.APP_NAME }}	Environment variables
vars	${{ vars.AWS_REGION }}	GitHub configuration variables
secrets	${{ secrets.API_TOKEN }}	Encrypted secrets
steps	${{ steps.x.outputs.y }}	Outputs from earlier step
needs	${{ needs.build.outputs.image }}	Outputs from dependency job
matrix	${{ matrix.python }}	Current matrix value
runner	${{ runner.os }}	Runner information

5. Variables, Secrets and Environment Variables

Workflow-level env

```
env:
  APP_NAME: devsecops-app
  PYTHON_VERSION: "3.12"
```

Job-level env

```
jobs:
  deploy:
    env:
      AWS_REGION: us-east-1
```

Step-level env

```
- name: Build
  env:
    IMAGE_TAG: ${{ github.sha }}
  run: echo "$IMAGE_TAG"
```

vars vs secrets

Type	Use for	Example
vars	Non-sensitive configuration	AWS region, repository name
secrets	Sensitive values	API tokens, passwords
env	Runtime variables inside workflow	IMAGE_TAG, APP_NAME

```
aws-region: ${{ vars.AWS_REGION }}
password: ${{ secrets.DATABASE_PASSWORD }}
```

Never put AWS access keys directly in YAML. For AWS deployments, prefer GitHub OIDC.

6. Step Outputs and Job Outputs

Create a step output

```
- name: Export Image URI
  id: image
  run: |
    IMAGE_URI="123456789012.dkr.ecr.us-east-1.amazonaws.com/app:${{ github.sha }}"
    echo "image=$IMAGE_URI" >> "$GITHUB_OUTPUT"
```

Read step output

```
${{ steps.image.outputs.image }}
```

Promote step output to job output

```
jobs:
  build:
    outputs:
      image: ${{ steps.image.outputs.image }}

    steps:
      - id: image
        run: echo "image=my-image:${{ github.sha }}" >> "$GITHUB_OUTPUT"
```

Use output from another job

```
jobs:
  deploy:
    needs: build

    steps:
      - run: echo "${{ needs.build.outputs.image }}"
```

7. uses vs run

uses - call an Action

```
- name: Checkout
  uses: actions/checkout@v4

- name: Setup Python
  uses: actions/setup-python@v5
  with:
    python-version: "3.12"
```

run - execute shell commands

```
- name: Run Tests
  run: |
    pip install -r app/requirements.txt
    pytest app/tests/
```

with - pass inputs to an Action

```
- uses: actions/upload-artifact@v4
  with:
    name: test-results
    path: reports/
```

working-directory

```
- name: Terraform Validate
  working-directory: terraform
  run: terraform validate
```

Shell selection

```
- name: Bash Example
  shell: bash
  run: echo "Linux shell"

- name: PowerShell Example
  shell: pwsh
  run: Write-Host "PowerShell"
```

8. Artifacts

Artifacts move files between jobs or preserve reports after a workflow completes.

Upload

```
- name: Upload Terraform Plan
  uses: actions/upload-artifact@v4
  with:
    name: terraform-plan
    path: |
      terraform/tfplan
      terraform/tfplan.txt
      terraform/tfplan.json
```

Download

```
- name: Download Terraform Plan
  uses: actions/download-artifact@v4
  with:
```

```
name: terraform-plan
path: terraform
```

Important: artifacts are files; outputs are small strings. Use the right mechanism for the right data.

9. Complete Python CI Example

```
name: Python CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install Dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r app/requirements.txt

      - name: Run Tests
        run: pytest app/tests/
```

10. Docker CI Example

```
jobs:
  docker:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Build Image
        run: |
          docker build             -f docker/Dockerfile             -t aws-devsecops-app:${{ github.sha }}             .

      - name: Run Container
        run: |
          docker run -d             --name devsecops-app             -p 5000:5000             aws-devsecops-app:${{ github.sha }}

      - name: Health Check
        run: |
          for i in {1..10}; do
            if curl --fail http://localhost:5000/health; then
              exit 0
            fi
            sleep 2
          done
          exit 1

      - name: Cleanup
        if: always()
        run: |
          docker stop devsecops-app || true
          docker rm devsecops-app || true
```

11. Trivy Security Scanning

Repository scan

```
- name: Trivy Repository Scan
  uses: aquasecurity/trivy-action@master
  with:
    scan-type: fs
    scan-ref: .
    scanners: vuln,secret
    severity: HIGH,CRITICAL
    ignore-unfixed: true
    exit-code: 1
```

Container scan

```
- name: Trivy Image Scan
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: aws-devsecops-app:${{ github.sha }}
    format: table
    severity: HIGH,CRITICAL
    ignore-unfixed: true
    exit-code: 1
```

Generate report without blocking

```
- name: Generate Trivy JSON Report
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: aws-devsecops-app:${{ github.sha }}
    format: json
    output: trivy-report.json
    severity: UNKNOWN,LOW,MEDIUM,HIGH,CRITICAL
    exit-code: 0
```

12. Terraform + Checkov

```
jobs:
  terraform:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: hashicorp/setup-terraform@v4

      - name: Terraform Format
        working-directory: terraform
        run: terraform fmt -check -recursive

      - name: Terraform Init for Validation
        working-directory: terraform
        run: terraform init -backend=false -input=false

      - name: Terraform Validate
        working-directory: terraform
        run: terraform validate

      - name: Checkov
        uses: bridgecrewio/checkov-action@v12
        with:
          directory: terraform
          framework: terraform
          quiet: true
```

```
soft_fail: false
```

13. AWS OIDC Authentication

Recommended pattern for GitHub Actions to AWS: short-lived OIDC credentials instead of stored AWS access keys.

```
permissions:
  id-token: write
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - uses: aws-actions/configure-aws-credentials@v6
        with:
          role-to-assume: ${{ vars.AWS_ROLE_ARN }}
          aws-region: ${{ vars.AWS_REGION }}

      - name: Verify AWS Identity
        run: aws sts get-caller-identity
```

OIDC requirement: the AWS IAM role trust policy must trust GitHub's OIDC provider and restrict the expected repository/branch.

14. Amazon ECR Pattern

```
- name: Login to Amazon ECR
  id: login-ecr
  uses: aws-actions/amazon-ecr-login@v2

- name: Build Image
  env:
    REGISTRY: ${{ steps.login-ecr.outputs.registry }}
    REPOSITORY: ${{ vars.ECR_REPOSITORY }}
    TAG: ${{ github.sha }}
  run: |
    docker build       -f docker/Dockerfile       -t "$REGISTRY/$REPOSITORY:$TAG"       .

- name: Push Image
  env:
    REGISTRY: ${{ steps.login-ecr.outputs.registry }}
    REPOSITORY: ${{ vars.ECR_REPOSITORY }}
    TAG: ${{ github.sha }}
  run: docker push "$REGISTRY/$REPOSITORY:$TAG"
```

Idempotent immutable-tag check

```
- name: Check if Image Exists
  id: image-check
  env:
    REPOSITORY: ${{ vars.ECR_REPOSITORY }}
    TAG: ${{ github.sha }}
  run: |
    if aws ecr describe-images       --repository-name "$REPOSITORY"       --image-ids imageTag="$TAG" >/dev/null 2>&1; then
      echo "exists=true" >> "$GITHUB_OUTPUT"
    else
      echo "exists=false" >> "$GITHUB_OUTPUT"
    fi
```

15. Terraform Plan -> Artifact -> Apply Pattern

Plan job

```
terraform-plan:
  needs: push-ecr
  runs-on: ubuntu-latest

  steps:
    - uses: actions/checkout@v4

    - uses: hashicorp/setup-terraform@v4

    - name: Terraform Init
      working-directory: terraform
      run: terraform init -reconfigure -input=false

    - name: Terraform Plan
      working-directory: terraform
      env:
        CONTAINER_IMAGE: ${{ needs.push-ecr.outputs.image }}
      run: |
        terraform plan           -var="container_image=$CONTAINER_IMAGE"           -out=tfplan           -input=false

    - uses: actions/upload-artifact@v4
      with:
        name: terraform-plan
        path: terraform/tfplan
```

Apply job

```
terraform-apply:
  needs: terraform-plan
  runs-on: ubuntu-latest

  steps:
    - uses: actions/checkout@v4

    - uses: hashicorp/setup-terraform@v4

    - working-directory: terraform
      run: terraform init -reconfigure -input=false

    - uses: actions/download-artifact@v4
      with:
        name: terraform-plan
        path: terraform

    - working-directory: terraform
      run: terraform apply -auto-approve -input=false tfplan
```

Why use a saved plan? The exact plan reviewed/generated in the plan job is what the apply job executes.

16. Concurrency

```
concurrency:
  group: devsecops-${{ github.ref }}
  cancel-in-progress: false
```

Useful for Terraform deployments. A second run on the same branch waits instead of interrupting an in-progress infrastructure change.

17. Matrix Builds

```
jobs:
  test:
    strategy:
      matrix:
        python:
          - "3.11"
          - "3.12"
          - "3.13"

    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python }}

      - run: pytest
```

A matrix creates multiple job executions from the same definition.

18. Caching

Python pip cache

```
- uses: actions/setup-python@v5
  with:
    python-version: "3.12"
    cache: pip
    cache-dependency-path: app/requirements.txt
```

General cache pattern

```
- uses: actions/cache@v4
  with:
    path: ~/.cache/example
    key: ${{ runner.os }}-${{ hashFiles('**/requirements.txt') }}
```

19. GitHub Environments and Deployment Protection

```
jobs:
  deploy-production:
    environment: production
    runs-on: ubuntu-latest

    steps:
      - run: echo "Deploy production"
```

GitHub Environments can hold environment-specific variables/secrets and, depending on repository/account settings, deployment protection rules such as reviewers.

20. Permissions: Least Privilege

Read-only repository

```
permissions:
  contents: read
```

OIDC to AWS

```
permissions:
  contents: read
  id-token: write
```

Avoid broad permissions unless the workflow actually needs them.

21. Timeout, Continue-on-error and Retry Thinking

```
jobs:
  test:
    timeout-minutes: 15
    runs-on: ubuntu-latest

    steps:
      - name: Optional Diagnostic
        continue-on-error: true
        run: some-diagnostic-command
```

`continue-on-error: true` means the step may fail without failing the entire job. Use it for optional diagnostics, not security gates.

22. GitHub Step Summary

```
- name: Deployment Summary
  run: |
    {
      echo "# Deployment"
      echo
      echo "| Item | Value |"
      echo "|---|---|"
      echo "| Commit | \`${{ github.sha }}\` |"
      echo "| Status | Success |"
    } >> "$GITHUB_STEP_SUMMARY"
```

23. Reusable Workflow Basics

Reusable workflow

```
name: Reusable Terraform Validate

on:
  workflow_call:
    inputs:
      directory:
        required: true
        type: string

jobs:
  validate:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v4
      - working-directory: ${{ inputs.directory }}
        run: |
          terraform init -backend=false
          terraform validate
```

Call it

```
jobs:
  terraform:
    uses: ./.github/workflows/terraform-validate.yml
    with:
      directory: terraform
```

24. Common Expression Functions

Function	Example	Use
contains()	contains(github.ref, 'main')	Check text/list
startsWith()	startsWith(github.ref, 'refs/tags/')	Tag checks
endsWith()	endsWith(github.ref, '-prod')	Suffix checks
format()	format('app-{0}', github.sha)	Construct strings
join()	join(matrix.targets, ',')	Join array values
toJSON()	toJSON(github)	Debug a context
fromJSON()	fromJSON('true')	Convert JSON text to value
hashFiles()	hashFiles('**/requirements.txt')	Cache keys

Debug a context carefully

```
- name: Debug GitHub Context
  run: echo '${{ toJSON(github) }}'
```

Do not dump secrets or sensitive contexts into logs.

25. Cloud/DevSecOps Job Ordering Pattern

```
jobs:
  test:
    ...

  repository-security:
    needs: test
    ...

  docker:
    needs:
      - test
      - repository-security
    ...

  terraform:
    needs:
      - test
      - repository-security
    ...

  push-ecr:
    needs:
      - docker
      - terraform
    ...

  terraform-plan:
    needs: push-ecr
    ...

  terraform-apply:
    needs: terraform-plan
    ...

  verify:
    needs: terraform-apply
    ...
```

Think of `needs` as the dependency graph for your delivery pipeline.

26. Failure Diagnostics Pattern

```
- name: Collect Failure Diagnostics
  if: failure()
  run: |
    mkdir -p diagnostics

    aws ecs describe-services       --cluster "$CLUSTER"       --services "$SERVICE"       > diagnostics/service.json || true

    aws logs filter-log-events       --log-group-name "$LOG_GROUP"       --limit 100       > diagnostics/logs.json || true

- name: Upload Diagnostics
  if: failure()
  uses: actions/upload-artifact@v4
  with:
    name: deployment-diagnostics
    path: diagnostics/
```

27. Most Common GitHub Actions YAML Mistakes

Problem	Bad / Symptom	Fix
Wrong indentation	Unexpected YAML parsing error	Align keys and list items consistently
Missing - before step	steps object invalid	Every step starts with '-'
uses and run together	Invalid step	A step normally uses one or the other
Wrong output path	steps.X.outputs is empty	Give producer step an id
Forgot job outputs	needs.job.outputs empty	Map step output under jobs.<job>.outputs
Secret in echo	Credential leaks to logs	Never print secrets
No id-token permission	OIDC role assumption fails	Add id-token: write
Old Terraform saved plan	Saved plan is stale	Regenerate plan after state changes
Immutable ECR duplicate tag	Tag cannot be overwritten	Check image before pushing
Bad if expression	Job unexpectedly skips	Check ref/event strings carefully
Failure diagnostics skipped	No logs after failure	Use if: failure() or always()
Terraform overlaps	State conflicts/races	Use concurrency

Common GitHub ref values

Situation	Value
main branch	refs/heads/main
develop branch	refs/heads/develop
tag v1.0.0	refs/tags/v1.0.0

Main-only deployment condition

```
if: github.ref == 'refs/heads/main' && github.event_name == 'push'
```

28. Copy/Paste Mini Templates

Checkout

```
- uses: actions/checkout@v4
```

Python

```
- uses: actions/setup-python@v5
  with:
    python-version: "3.12"
```

Terraform

```
- uses: hashicorp/setup-terraform@v4
```

AWS OIDC

```
- uses: aws-actions/configure-aws-credentials@v6
  with:
    role-to-assume: ${{ vars.AWS_ROLE_ARN }}
    aws-region: ${{ vars.AWS_REGION }}
```

ECR Login

```
- id: login-ecr
  uses: aws-actions/amazon-ecr-login@v2
```

Upload Artifact

```
- uses: actions/upload-artifact@v4
  with:
    name: report
    path: report.json
```

Download Artifact

```
- uses: actions/download-artifact@v4
  with:
    name: report
    path: .
```

Always cleanup

```
- name: Cleanup
  if: always()
  run: ./cleanup.sh
```

Failure-only diagnostics

```
- name: Diagnostics
  if: failure()
  run: ./collect-diagnostics.sh
```

29. Mental Model for Writing Any Workflow

When you create a workflow from scratch, answer these questions in order:

#	Question	YAML area
1	What starts it?	on
2	What permissions are required?	permissions
3	Can runs overlap?	concurrency
4	What are the major stages?	jobs
5	Which jobs depend on others?	needs
6	What runner does each job need?	runs-on
7	What action/command does each step run?	uses / run
8	What configuration is needed?	with / env / vars
9	What data must move between steps/jobs?	outputs / artifacts
10	When should something run?	if
11	How will failures be debugged?	failure() diagnostics
12	What proves deployment succeeded?	health/verification steps

Recommended DevSecOps order

```
Test
  -> Security Scan
  -> Build
  -> Container Scan
  -> IaC Validate
  -> IaC Security Scan
  -> Authenticate
  -> Publish Artifact
  -> Plan
  -> Safety Gate
  -> Apply
  -> Verify
  -> Observe / Diagnose
```

Best habit: keep workflow jobs small enough that the job name tells you exactly which stage failed.

30. Quick Final Reference

Need	Use
Run after another job	needs
Only main branch	if: github.ref == 'refs/heads/main'
Run even after failure	if: always()
Run only after failure	if: failure()
Repository variable	${{ vars.NAME }}
Secret	${{ secrets.NAME }}
Current commit	${{ github.sha }}

Need	Use
Step output	${{ steps.ID.outputs.NAME }}
Job output	${{ needs.JOB.outputs.NAME }}
Write output	echo "name=value" >> "$GITHUB_OUTPUT"
Workflow summary	>> "$GITHUB_STEP_SUMMARY"
Pass files between jobs	upload-artifact / download-artifact
AWS without static keys	OIDC + configure-aws-credentials
Stop parallel deploys	concurrency

Keep this PDF beside your editor while building workflows. Once these patterns become familiar, most GitHub Actions YAML is just combinations of the same building blocks.